

qnx-micro System Architecture

A C++26 microkernel that teaches you QNX by reading the code

Rajeshkumar Venugopal
Third Buyer Advisory

April 2026

1 Why QNX

Every RTOS you've seen is a lie. VxWorks, FreeRTOS, Zephyr — they call themselves real-time, but they're monolithic. The scheduler, the filesystem, the network stack, the device drivers: all in one address space. One bad pointer in a driver takes down the kernel.

QNX Neutrino is different. The kernel does three things: pass messages, schedule threads, manage memory. Everything else runs in user space. A driver crash doesn't take down the kernel. It takes down the driver. The kernel restarts it.

2 What flips from monolithic to micro

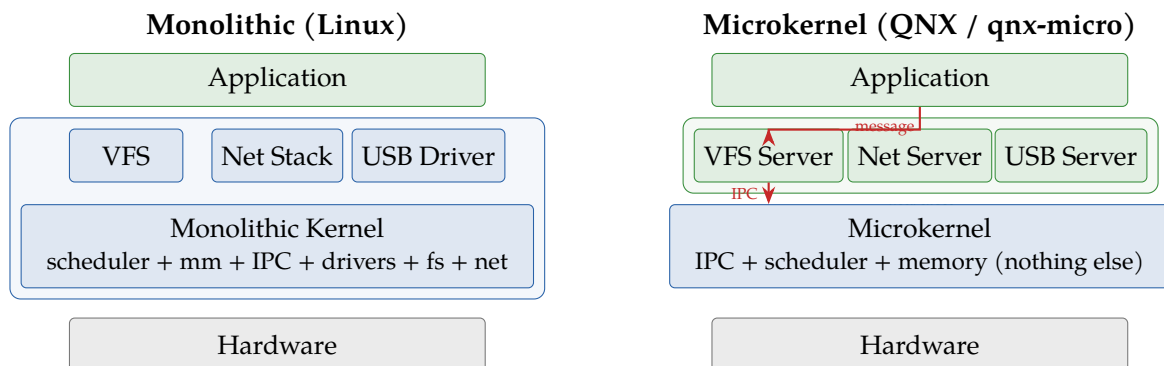


Figure 1: A driver crash in the monolithic kernel panics the system. In the microkernel, the driver is restarted.

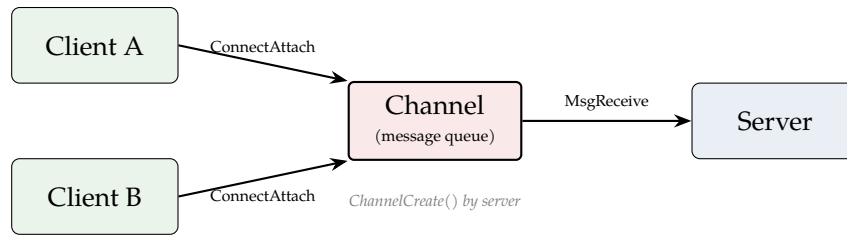


Figure 2: Multiple clients connect to one server channel.

3 The message-passing model

3.1 Channels and connections

3.2 The send/receive/reply rendezvous

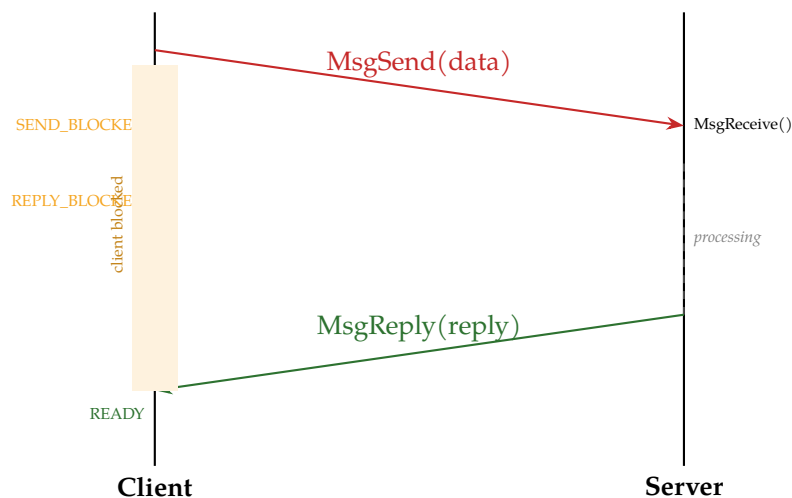


Figure 3: Synchronous rendezvous: client blocks from send until reply.

Three blocking states:

1. SEND_BLOCKED: sent, waiting for server to receive.
2. REPLY_BLOCKED: server received, waiting for reply.
3. RECEIVE_BLOCKED: server waiting, no senders.

Synchronous IPC gives implicit flow control (client can't flood server), implicit priority inheritance, and zero-copy potential.

Pulses are the async escape: 8-byte notifications queued without blocking the sender.

3.3 Priority inversion

Synchronous IPC creates a priority inversion hazard: a high-priority client blocks on `MsgSend` to a low-priority server. The server cannot run because medium-priority threads preempt it.

The fix is implicit priority inheritance. When a thread's wait queue changes, the scheduler recomputes:

$$\text{effective_priority} = \max(\text{declared}, \max(\text{waiter priorities}))$$

The server is promoted to the highest waiter's priority for the duration of the rendezvous. Z3 proof A9 verifies that the effective priority is always $\geq$ the declared priority.

4 Thread state machine

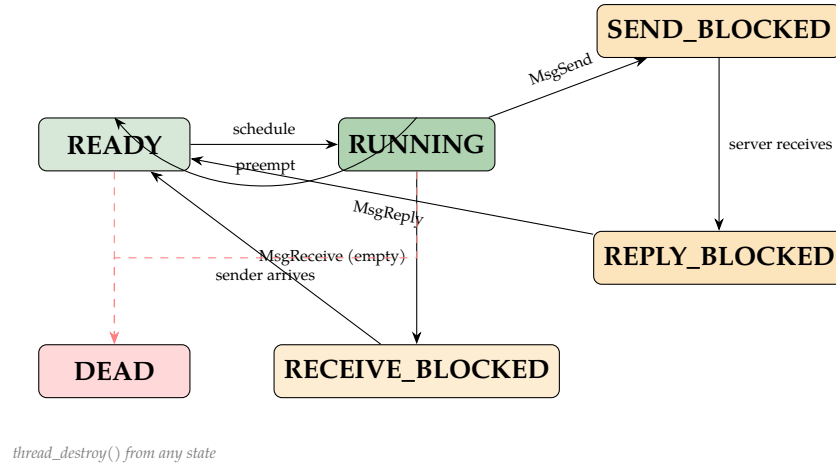


Figure 4: Thread state machine. Z3 proves 9 invariants hold for all states.

5 The scheduler

Fixed-priority preemptive. 256 levels. No timeslicing.

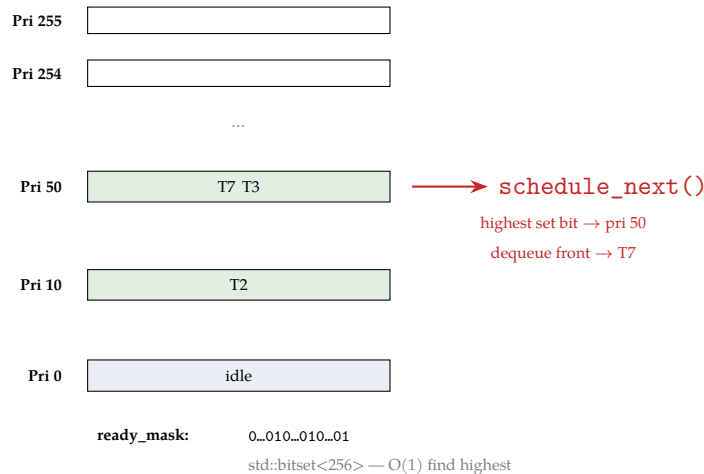


Figure 5: Priority queues with bitmask for O(1) dispatch.

A thread runs until it blocks or a higher-priority thread becomes ready. On `tick()`, the scheduler checks for preemption.

6 Memory: capabilities

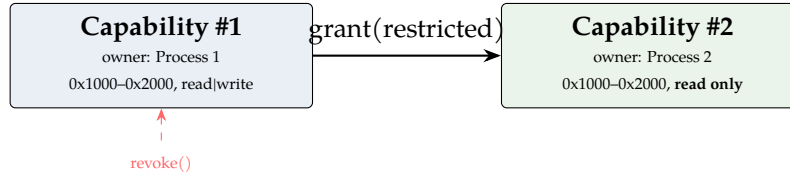


Figure 6: No shared memory without explicit capability grant.

Real backing store: `SharedBuffer` holds a fixed-capacity `std::array<std::byte, max_msg_size>`. Capabilities reference buffers by `BufferId`. `write_span` and `read_span` return `std::span` into the actual bytes. Two capabilities on the same buffer = zero-copy shared memory.

Grant creates a child capability with intersected (never escalated) permissions over a sub-region. Revoke cascades to all children.

Five capability invariants proved UNSAT by Z3: no escalation, acyclic chain, transitive restriction, same buffer, root has no ancestor.

7 Resource managers: drivers in user space

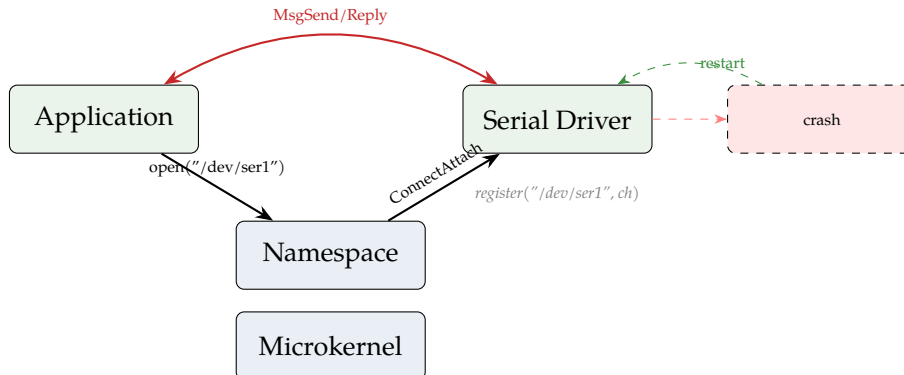


Figure 7: Driver crashes and restarts. System continues.

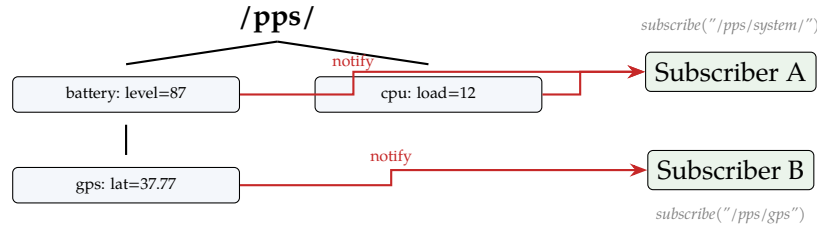


Figure 8: Publishers write. Subscribers get notified. No polling.

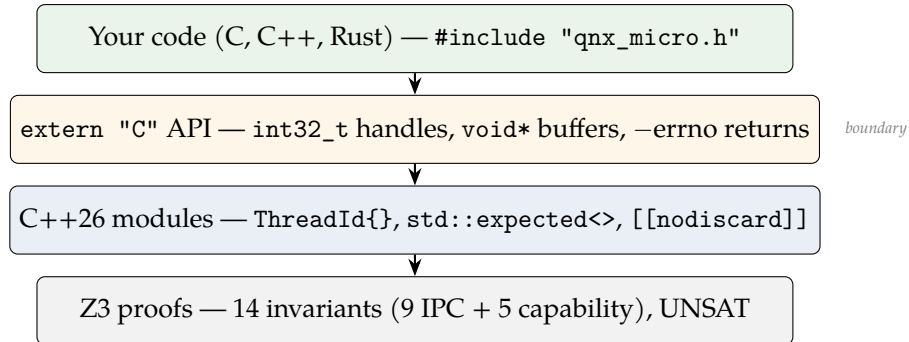


Figure 9: Strong types inside. C integers outside. Z3 underneath.

8 PPS: Persistent Publish/Subscribe

9 The extern “C” layer

10 Verification pyramid

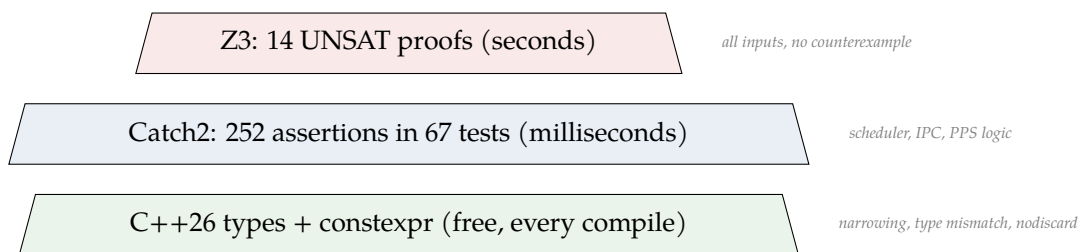


Figure 10: Each layer catches what the layer below cannot.

11 Module reference

Module	File	Purpose
qnx.types	types/types.cppm	Strong handles, enums, Result<T>.
qnx.scheduler	scheduler/scheduler.cppm	256-level O(1) preemptive scheduler.

qnx.ipc	ipc/ipc.cppm	Send/receive/reply rendezvous, pulses.
qnx.memory	memory/memory.cppm	Capability-based mmap/grant/revoke.
qnx.driver	driver/driver.cppm	Resource manager namespace.
qnx.pps	pps/pps.cppm	Persistent publish/subscribe.
qnx.kernel	kernel/kernel.cppm	Bootstrap, owns all subsystems.

12 Test reference

File	Tests	Covers
test_scheduler.cpp	13	priority, preemption, block/unblock, destroy, priority inversion
test_ipc.cpp	8	send/receive/reply cycle, pulse, errors
test_pps.cpp	14	publish, subscribe, notify, delete, list
test_memory.cpp	17	mmap, spans, grant restriction, sub-regions, shared buffer, revoke cascade,
test_api_compat.cpp	10	QNX Neutrino API signature compatibility
test_qnx_scenarios.cpp	5	openqnx IPC patterns
Total	67	252 assertions

13 Example: AudioManager

examples/audio_manager.cpp demonstrates the full stack. Producer allocates a 3840-byte PCM buffer, writes samples, grants a read-only capability to the /dev/audio driver, and sends the 4-byte capability ID via IPC. The driver calls read_span — zero copy from the producer’s buffer. PPS notifies the UI of state and track changes. Revoke kills driver access when playback ends.

Result: 3840 bytes transferred, zero copies, one IPC message, four PPS notifications. DMA not needed.

13.1 IPC benchmark

100k send/receive/reply round-trips, Apple M3, -O2:

- qnx-micro: **1.31 μ s** per round-trip.
- QNX Neutrino (600 MHz ARM): $\sim 2 \mu$ s.
- FreeRTOS context switch (Cortex-M4 @168 MHz): $\sim 7 \mu$ s.

14 extern “C” API

```
// IPC
int32_t qnx_channel_create(qnx_process_id owner);
int32_t qnx_connect_attach(qnx_channel_id ch, qnx_process_id pid);
int32_t qnx_msg_send(qnx_connection_id conn, const void* s, size_t sl,
                    void* r, size_t rl);
int32_t qnx_msg_receive(qnx_channel_id ch, void* buf, size_t len);
```

```

int32_t qnx_msg_reply(qnx_thread_id sender, const void* buf, size_t len);
int32_t qnx_msg_send_pulse(qnx_connection_id conn, int32_t code, int32_t val);

// Scheduler
int32_t qnx_thread_create(qnx_process_id pid, int32_t priority);
int32_t qnx_thread_destroy(qnx_thread_id tid);

// Memory
int64_t qnx_mmap(qnx_process_id pid, size_t length, int32_t perm);
int32_t qnx_munmap(qnx_process_id pid, int64_t base);

// Resource managers
int32_t qnx_register_resource(const char* path, qnx_channel_id ch,
                             qnx_process_id pid);
int32_t qnx_resolve_path(const char* path, qnx_process_id client_pid);

```

15 Convenience helpers

- `pid(n)`, `pri(n)` — shorthand constructors.
- `as_bytes(str)`, `as_string(span)` — buffer conversion.
- `perm_rw`, `perm_rx`, `perm_rwx` — permission combos.

16 Build

```

brew install llvm doxygen          # macOS
cmake -S . -B build \
  -DCMAKE_C_COMPILER=/opt/homebrew/opt/llvm/bin/clang \
  -DCMAKE_CXX_COMPILER=/opt/homebrew/opt/llvm/bin/clang++
cmake --build build                # build
cmake --build build --target test  # 67 tests, 252 assertions
cmake --build build --target doc-coverage # doxygen ratchet

```

17 License

BSD 2-Clause. Every source file carries SPDX-License-Identifier: BSD-2-Clause. Not affiliated with BlackBerry QNX.